

ALE 課程 6：自訂 IDoc 擴充 (Segment / Extension / 自訂訊息類型)

Lecture

ale02 ~ ale05 都在用標準的 IDoc 型別 (**MATMAS05**)。但真實專案幾乎一定會遇到「標準 IDoc 沒有我要的欄位」或「標準根本沒有這種業務訊息」——這題教怎麼自己定義 IDoc 的形狀。注意：這題只定義結構，不送資料；填值與送出是 ale07 (Outbound 客製化) 的事。

兩條擴充路線

	路線 A：Extension (擴充既有 Basic Type)	路線 B：自訂完整 Z 訊息類型
做法	在標準 Basic Type (如 MATMAS05) 的某個 Segment 底下，多掛你自己的 Segment	全新定義 Message Type + Basic Type + Segment，跟標準完全無關
用到的交易碼	WE31 (建 Segment) → WE30 (建 Extension) → WE82 (指派)	WE81 (建 Message Type) → WE31 → WE30 (建 Basic Type) → WE82 (指派)
適用情境	標準訊息大致夠用，只差幾個欄位	標準沒有對應訊息，或標準結構差太多
對標準的相依	高：標準 Basic Type 升級 (如 MATMAS05 → MATMAS06) 時，Extension 要跟著重新對應	無：完全獨立，升級不受影響
標準處理邏輯	可沿用標準的 Outbound / Inbound Function Module (新 Segment 靠 User-Exit 填值，見 ale07/ale08)	沒有任何標準處理邏輯，Outbound 組 IDoc、Inbound 過帳都要自己寫
本課程對應	本題 Part A；ale10 期末案例二	本題 Part B；ale09 期末案例一

取捨口訣：標準八成夠用選 Extension (省工，但綁著標準升級走)；標準沒有或差很多選自訂 (自由，但處理邏輯全部自己寫)。

Segment 的解剖：Segment Type、Segment Definition、版本

一個 Segment 在系統裡其實是兩層物件加上一份中繼資料——用系統上真實的標準 Segment **E1MARAM** 實測 (2026-09-21)：

- **Segment Type (E1MARAM)**：DDIC Structure，欄位全部是 **abap.char(n)** (如 **matnr**：**abap.char(18)**、**ersda**：**abap.char(8)**)。這就是 IDoc Data Record (**EDID4-SDATA** 純文字字串) 真正對應的扁平字元結構。
- **Segment Definition (E2MARAM)**：另一個 DDIC Structure，欄位引用真正的 **Data Element** (**matnr**：**matnr**；**lvorm**：**lvoma**；**mtart**：**mtart**)。這才是「業務欄位的語意定義」。
- **中繼資料表**：**EDISDEF** (Segment 的版本清單：**E1MARAM** 有 **000 ~ 008** 共 9 個版本，各自指向 **E2MARAM**、**E2MARAM001...E2MARAM008**，附 **RELEASED** 釋出版本與 **FIELDNUM** 欄位數) + **EDSAPPL** (每個欄位的 **ROLLNAME** = Data Element、**EXPLENG** = 匯出長度)。

這修正並細化了 **ale01** 的說法：ale01 說「Segment Type 本質是一張 DDIC Structure」——精確講，**E1xxx** (Type) 是扁平字元版，**E2xxx###** (Definition) 才是引用 **Data Element** 的版本，兩者都是 DDIC Structure。你在 **WE31** 建一個 Segment **Z1ALE06**，系統會自動生成這兩層 (**Z1ALE06** 與 **Z2ALE06000**) 並寫入 **EDISDEF** / **EDSAPPL**。

⚠ 因此 **Claude** 不能代建 Segment (**ale06** 待驗證項目已有結論)：即使用 **sap_create_object** / **sap_set_source** (**objectType=STRU**) 建出一張同名 Structure，也不會寫入 **EDISDEF** / **EDSAPPL**，**WE30** 認不得它是 Segment；更糟的是會讓 **WE31** 之後建同名 Segment 時撞名。所以 Segment、Extension、Basic Type、Message Type 的建立一律走 **GUI**，**Claude** 負責事後用字典表 + 讀取生成的 Structure 來驗證。請不要事先建立名為 **Z1ALE06** / **Z2ALE06000** 的任何物件。

命名規則與欄位設計 (呼應課程 Schema 硬性規則)

- Segment Type 在客戶命名空間必須以 **Z1** 開頭 (**Z1ALE06**)；系統自動生成的 Definition 是 **Z2ALE06000** (尾碼三位數是版本號)。
- Segment** 欄位一律引用標準 **Data Element** (**WE31** 的 **Data element** 欄位，就是 **EDSAPPL-ROLLNAME** 記錄的東西)，不要自建。這題的欄位都已用系統實測確認對應的標準表欄位與 Data Element：**MARA-SAISJ**→**SAISJ** (Season Year, 4 碼)、**MARA-DATAB**→**DATAB** (Valid-From Date)、**EKKO-EBELN**→**EBELN**、**EKKO-BUKRS**→**BUKRS**、**EKKO-LIFNR**→**ELIFN** (⚠ 採購文件裡的供應商欄位 Data Element 是 **ELIFN**，不是 **LFA1** 用的 **LIFNR**)、**EKPO-EBELP**→**EBELP**。
- 物料號碼欄位要用 **MATNR18**，不是 **MATNR**：**EDSAPPL** 實測 **E1MARAM-MATNR** 的 **ROLLNAME** 是 **MATNR18** (**EXPLENG** = **0018**)——因為 S/4HANA 的 **MATNR** 是 40 碼，IDoc 為了相容舊格式，標準 Segment 一律用專門的 18 碼版本 **MATNR18**。自訂 Segment 要跟標準 Segment 一致，同樣用 **MATNR18**。

Segment 的階層與基數

IDoc 不是扁平的：**WE30** 裡每個 Segment 都要指定上層 Segment (**EDID4-PSGNUM** 記錄的就是這個父子關係) 以及最小 / 最大出現次數 (**Min/Max**)。例如訂單表頭 Segment 出現 1 次 (**1..1**)、底下掛的明細 Segment 出現 1 到 999 次 (**1..999**)；Extension 新增的 Segment 通常是選用的 (**0..1**，沒資料就不送)。

釋出 (Release) 與指派 (**WE82**)

Segment、Basic Type / Extension 建立後都是未釋出狀態，要各自執行 **Set Release** 才能被實際使用 (**EDISDEF-RELEASED**、**EDBAS-RELEASED**、**EDIMSG-RELEASED** 都有這個欄位)。最後用 **WE82** 把「Message Type ↔ Basic Type (+ Extension)」的對應關係登記進去 (**EDIMSG** 表，系統實測 **MATMAS** 已有 **MATMAS01** ~ **MATMAS06** 等多筆指派)——沒有這筆指派，**Message Type** 跟 **IDoc** 結構就沒有關聯。

學習目標

- 能分辨 Extension 與自訂完整 Z 訊息類型兩條路線的取舍，並針對情境選擇
- 能講出 Segment Type (**E1/Z1**，扁平字元) 與 Segment Definition (**E2/Z2###**，引用 Data Element) 的差異，以及 **EDISDEF** / **EDSAPPL** 各記錄什麼
- 知道為什麼 Segment 不能由 ADT 直接建立，以及建立後如何用字典表驗證
- 能完成 **WE31**→**WE30**→**WE82** 的建立流程 (Extension 與自訂訊息類型各一次)，並理解 Release 步驟
- 能正確選擇 Segment 欄位的 Data Element (含 **MATNR18**、**ELIFN** 這兩個容易選錯的標準例子)

事前準備

不需要新建 ABAP 原始碼物件，全部是 **SAP GUI** 操作（**WE31** / **WE30** / **WE81** / **WE82** / **WE60**）。建立時套件請選 **\$TMP**（不需傳輸請求）。系統已確認 **Z1ALE06**、**Z1ALE06H**、**Z1ALE06I**、**ZALE06**、**ZALE06_MATEXT**、**ZALE06_BT01** 等名稱目前都不存在，可以放心使用。

題目需求

Part A：Extension——替 **MATMAS05** 多掛一個 Segment

步驟 1：WE31 建立 Segment **Z1ALE06**

1. 交易碼 **WE31** → **Segment type** 填 **Z1ALE06** → **Create**
2. **Short description** 填 **ALE06 material extension**
3. 依序新增兩個欄位（Field name / Data element）：
 - **SAISJ** / **SAISJ**
 - **DATAB** / **DATAB**
4. 存檔（系統會提示生成 Segment Definition，接受預設名稱，預期為 **Z2ALE06000**）
5. 選單 **Edit** → **Set Release**（釋出這個 Segment）

步驟 2：WE30 建立 Extension **ZALE06_MATEXT**

1. 交易碼 **WE30** → **Object name** 填 **ZALE06_MATEXT** → 選 **Extension** → **Create**
2. 系統詢問連結的 Basic Type，填 **MATMAS05**
3. 在樹狀結構選取 **E1MARAM** 這個 Segment → **Create**（新增子節點）→ **Segment type** 填 **Z1ALE06**，**Minimum 0**、**Maximum 1**
4. 存檔，選單 **Edit** → **Set Release**（釋出這個 Extension）

步驟 3：WE82 指派

1. 交易碼 **WE82** → **New Entries**
2. **Message Type** **MATMAS**、**Basic type** **MATMAS05**、**Extension** **ZALE06_MATEXT**、**Release** 填你系統的版本（可參考該畫面既有 **MATMAS05** 那列的值）
3. 存檔

步驟 4：WE60 檢視結構

交易碼 **WE60** → 選 Basic Type **MATMAS05**、Extension **ZALE06_MATEXT** → **Execute**，展開樹狀結構，確認 **E1MARAM** 底下出現了 **Z1ALE06**，並記下它的 Min/Max 與兩個欄位的長度。

Part B：自訂完整 Z 訊息類型的結構骨架（表頭 / 明細）

這題只建骨架，不送資料、不設定 **Partner Profile**（那是 ale07 的事）。

步驟 5：WE31 建兩個 Segment

- **Z1ALE06H**（表頭）：EBELN/EBELN、BUKRS/BUKRS、ELIFN/ELIFN
- **Z1ALE06I**（明細）：EBELP/EBELP、MATNR18/MATNR18

（⚠ **Field name** 必須與 **Data element** 同名，且 **Segment** 名稱、欄位順序照上面寫的——ale07 / ale08 的 ABAP 程式碼直接引用 **z1ale06h-ebeln**、**z1ale06i-matnr18** 這類欄位名稱，名稱不同程式碼就編譯不過。Part A 的 **Z1ALE06**（**SAISJ** / **DATAB**）同理。）兩個都存檔後各自 **Set Release**。

步驟 6：WE81 建立 Message Type

1. 交易碼 WE81 → 進入變更模式 → **New Entries**
2. **Message Type** 填 ZALE06，**Description** 填 ALE06 custom purchase message
3. 存檔

步驟 7：WE30 建立 Basic Type ZALE06_BT01

1. 交易碼 WE30 → **Object name** ZALE06_BT01 → 選 **Basic type** → **Create**
2. 建立根節點：**Create** → Z1ALE06H，Min 1、Max 1
3. 選取 Z1ALE06H → **Create** (子節點) → Z1ALE06I，Min 1、Max 999
4. 存檔，**Set Release**

步驟 8：WE82 指派

New Entries：Message Type ZALE06、Basic type ZALE06_BT01 (Extension 留空)、Release 同上，存檔。

步驟 9：WE60 檢視

確認 ZALE06_BT01 的樹狀結構是「Z1ALE06H (1..1) → 底下 Z1ALE06I (1..999)」。

參考答案 (驗證方式)

全部建完後回報 (或直接說「建好了」)，我會用以下已實測可查的字典表逐項驗證 (表名與欄位皆於 2026-09-21 對本系統確認存在)：

- SELECT SEGTyp, VERSION, SEGDEF, RELEASED, FIELDNUM FROM EDISDEF WHERE SEGTyp IN ('Z1ALE06','Z1ALE06H','Z1ALE06I')——三個 Segment 都已生成 Definition 並釋出
- SELECT SEGTyp, POS, FIELDNAME, ROLLNAME, EXPLENG FROM EDSAPPL WHERE SEGTyp LIKE 'Z1ALE06%' ORDER BY SEGTyp, POS——逐欄核對 ROLLNAME 是否確實是標準 Data Element (SAISJ / DATAB / EBELN / BUKRS / ELIFN / EBELP / MATNR18)
- SELECT MESTyp, IDOCTyp, CIMTyp, RELEASED FROM EDIMSG WHERE MESTyp IN ('MATMAS','ZALE06')——確認 MATMAS + MATMAS05 + ZALE06_MATEXT、以及 ZALE06 + ZALE06_BT01 兩筆指派都存在
- SELECT IDOCTyp, RELEASED FROM EDBAS WHERE IDOCTyp IN ('ZALE06_BT01','ZALE06_MATEXT')——Basic Type / Extension 本身的釋出狀態
- TADIR + sap_get_source(objectType=STRU)：確認系統生成的 Z1ALE06 (全 abap.char) 與 Z2ALE06000 (引用 Data Element) 兩層 Structure 的原始碼，跟上方 E1MARAM / E2MARAM 的模式一致

思考題

1. Extension 的 Segment 建好、WE82 也指派了，但如果你在 WE20 的 MATMAS Outbound Partner Profile 沒有填 Extension、Inbound 端的 Function Module 也沒有登記支援這個 Extension (WE57)，實際送一筆帶 Z1ALE06 的 IDoc 出去，你預期會在哪一段失敗？(提示：想想「定義結構」「送出」「Inbound 處理」是三個各自獨立要設定的環節，ale07/ale08 會實際處理)
2. MATMAS05 有一天被 SAP 升級成 MATMAS06，你的 Extension ZALE06_MATEXT 綁的是 MATMAS05。這對路線 A 與路線 B 的影響各是什麼？(提示：回顧比較表「對標準的相依」那一列)

3. 為什麼 SAP 要把 Segment 拆成 E1 (扁平字元) 與 E2 (引用 Data Element) 兩層，而不是只留一層？
(提示：想想 EDID4-SDATA 存的是什麼型別，以及開發者查欄位「這是什麼意思」時需要哪一層)

答案

本題全程為 SAP GUI 操作，沒有可快照的 ABAP 原始碼。完成後我會用上方查詢驗證並讀取系統生成的 Z1ALE06 / Z2ALE06000 Structure，屆時視需要把這幾個 Structure 快照到本目錄 (.stru.abap) 作為重建參考。